~45 min total

Jordan Rodgers

Staff Offensive Security Engineer — Vast Space

SLIDE COPY

AEGIS · THE FLEET · SOVEREIGN

A multi-tier autonomous security vision for Haven-1

jordan.rodgers@gmail.com · linkedin.com/in/jollyrodgers · github.com/jolly-rodgers · jordanrodgers.dev

I build offensive security programs that find what adversaries would actually exploit.

AGENDA — 45 MINUTES

- **Background & career arc** — 10 minutes
- **Project 1: AEGIS + THE FLEET** — AI-driven SPARTA emulation framework built against Haven-1. Deep dive. — 20 minutes
- **SOVEREIGN** — the multi-year autonomous security program vision — 5 minutes
- **Brilliant idea: AIRAG** — Autonomous Incident Response & AI Guardian — 5 minutes
- **Questions** — 5 minutes

10+ years in offensive security — from tradecraft to founding programs.

CAREER TIMELINE

Apr 2015 – Jun 2020

Principal Security Engineer

Black Hat Defense — Brooklyn, NY

Engineered custom C2 infrastructure to simulate sophisticated multi-phase APT campaigns. Performed reverse engineering and malware tradecraft analysis to replicate nation-state TTPs. Institutionalized MITRE ATT&CK mapping across all offensive engagements — standardized framework for technique emulation and defensive gap analysis.

Sep 2021 – Present

Principal Offensive Security Engineer — Automation & Infrastructure

TechO Services — Brooklyn, NY (Hybrid)

Architected and scaled a continuous control validation ecosystem — transforming manual penetration testing into a high-fidelity automated security baseline program. Directed multi-million dollar vendor security budgets. Pioneered Detection-as-Code integration: automatically converting offensive findings into hardened detection logic, eliminating silent detection failures across global infrastructure.

Next

Staff Offensive Security Engineer

Vast Space — Long Beach, CA

Build the program from scratch. AEGIS. THE FLEET. SOVEREIGN. AIRAG.

Technical foundation

Adversary simulation

Metasploit | Custom C2 | Cobalt Strike | YAML orchestration | MITRE ATT&CK | SPARTA v3.2

AI / LLM red teaming

Garak | PyRIT | Promptmap | LLM-Fuzzer | Claude API | MITRE ATLAS | OWASP LLM Top 10

Cloud & infrastructure

AWS | GCP | Azure | Docker | Kubernetes | Terraform | Shodan

Languages & automation

Python | Go | Bash | PowerShell | OPA/Rego | Sigma

Hardware & embedded systems depth is an area I am actively building toward — the SOVEREIGN and AIRAG vision specifically calls for RTOS-level work, and I am treating that as a Year 1 priority. Vast has indicated this is an area where small gaps can be bridged on the job.

Where I have operated

Enterprise & Active Directory

Full kill-chain adversary emulation at TechO — custom C2, credential attacks, lateral movement, domain compromise. Built the continuous validation program that replaced point-in-time pentests.

Cloud-native & AWS

Cloud offensive operations across AWS, GCP, and Azure. IAM privilege escalation, cross-account lateral movement, container escape. AEGIS runs its attack chains against AWS-modeled infrastructure.

AI & LLM systems

Red teaming LLM deployments using Garak, PyRIT, and custom harnesses. Prompt injection, RAG data leakage, model inversion, data poisoning. Adversarial simulation framework published on GitHub.

Detection-as-Code

Pioneered the pipeline at TechO that automatically converted offensive findings into SIEM detection logic. Eliminated silent detection failures across global infrastructure. AEGIS does this automatically.

Why Vast

- **Founding red team hire at the only operational commercial space station company** — I want to build the program, not inherit it. Every decision I make in year one becomes the foundation every subsequent station inherits.
- **The attack surface is unlike anything else in the industry** — OT, IT, AI, and life-safety systems converging in a single environment. No playbook exists. Whoever takes this role writes it.
- **Haven-1 launching makes the threat model live and real** — the Space ISAC recorded a 118% surge in space-related cyber incidents in 2025. Nation-state adversaries are already watching. The window to build proactive capability before the first incident is closing.

PROJECT 1

AEGIS + THE FLEET

AI-Driven SPARTA Emulation & Gap Identification System
+ Autonomous Multi-Agent Security Platform

WHAT THESE TWO THINGS ARE

AEGIS is Fleet General v1.0 — a single AI reasoning agent. Claude reasoning over SPARTA v3.2, generating a real attack chain against Haven-1's architecture, identifying every gap, producing working fix code. It proves the concept works.

THE FLEET is the full multi-agent platform built on the same engine — 14 specialized agents orchestrated by the General. Attack and defense run simultaneously. The General adapts mid-operation. POC built and running.

The problem I was solving

THE GAP

No existing adversary emulation framework simultaneously covered spacecraft-specific TTPs (SPARTA), AI/ML system attack surface, agentic automation, and working defensive fix generation in a single pipeline — specifically modeled against a crewed space station.

REQUIREMENTS

- **SPARTA-native** — every TTP, countermeasure, and gap uses real SPARTA v3.2 identifiers
- **Haven-1 specific** — a digital twin modeling the actual system types Haven-1 runs: SSO, CI/CD, OBC, CCSDS uplink, Modbus life support
- **Working output** — OPA policies, Sigma rules, and Terraform configs deployable same day. Not a findings list.
- **Crew safety weighting** — gaps ranked by crew safety impact, not CVSS score
- **Solo operable** — designed to multiply a single operator’s capability on day one

AEGIS architecture

PIPELINE — FIVE STAGES

INPUT
SPARTA v3.2 knowledge base (847 techniques · 9 tactics) + Haven-1 architecture context + scenario.yaml

↓

AI MISSION PLANNER — CLAUDE API
Reasons over SPARTA KB. Selects TTPs for that actor against that target. Generates sequenced attack chain as JSON.

↓

GO EXECUTION RUNNER — DOCKER LAB (6 CONTAINERS)
Reads chain.json. Executes each step against the isolated Haven-1 digital twin. Zero SIEM noise. Zero operational impact.

↓

AI GAP ANALYZER — CLAUDE API
Identifies missing SPARTA countermeasures. Maps each to NIST 800-53. Scores by crew safety weight. Generates working fix files.

↓

OUTPUT
debrief.md · heatmap.html · roadmap.json · OPA/Conftest policies · Sigma detection rules · Terraform configs — deploy same day

7/7	23	47s
Kill chain steps pass	Missing countermeasures identified	End-to-end runtime

The Haven-1 digital twin

Six containers. Two networks. One command.

Ground network

- **haven-sso** — Okta-style SSO. Weak session handling. Phishable tokens.
- **haven-cicd** — Jenkins pipeline. Injectable build steps. Real artifact signing.
- **haven-uplink** — CCSDS telecommand endpoint. Logs all commands. Replay surface.

Space network

- **haven-obc** — Onboard computer. Signed firmware. Modbus register access. Implant simulation.
- **haven-lifesupport** — Life support OT (Modbus). O2, CO2, pressure registers. Spoofable via implant.
- **haven-comms** — Downlink simulator. Grafana + Loki live telemetry.

ONE COMMAND STARTS EVERYTHING

```
docker compose up -d
```

The network boundary between ground and space is real. The attack has to cross it — exactly the way a real adversary would.

KEY SLIDE

Nation-state APT. 7 steps. Zero failures.

KILL CHAIN — NATION_STATE_CAPTURE SCENARIO

STEP	SPARTA	WHAT HAPPENED	RESULT
01	IA-0007	Ground identity compromise. Engineer spearphished. Session token captured in 0.3s. No exploit required.	PASS
02	IA-0009.02	CI/CD pipeline injection. Build artifact poisoned. Hidden in binary test file — SAST sees nothing. Signed and ships.	PASS
03	EX-0009	OBC remote code execution. Implant installed. Persists across reboot via firmware write.	PASS
04	REC-0005	Command intercept. 1,247 uplink commands captured. Full control language mapped. Entirely passive. No alerts.	PASS
05	EX-0014.03	Life support sensor spoofing. O2 reporting 20.9% — actual: 14.2%. Ground sees nominal. Crew unaware.	PASS CREW HIGH
06	EX-0001.01	Station maneuver + comms cutoff. Commands replayed. Station moves. Downlink disrupted. Mission control: blind.	PASS
07	IMP-0002	Station capture. Foreign asset achieves rendezvous. Docking system compromised via implant. Hatch opens. 4 crew aboard.	PASS CREW CRIT

O2 AT 14.2% — CREW UNAWARE — GROUND OPS BLIND

KEY SLIDE

23 gaps. One fix collapses the chain.

TOP COUNTERMEASURES — RANKED BY CREW SAFETY IMPACT

RANK	CM ID	DESCRIPTION	BLOCKS	WEIGHT
1	CM0034	SBOM + build provenance verification	Steps 2+3	10
2	CM0001	MFA on ground ops SSO — FIDO2 hardware keys	Step 1	9
3	CM0019	Secure boot + code signing on OBC	Step 3	10
4	CM0041	Independent sensor validation + physical analog gauges	Step 5	10
5	CM0007	Command authentication MAC + replay detection	Steps 4+6	9

Priority: `chains_blocked × crew_safety_weight` · life_support=10 · propulsion=9 · docking=8 · comms=7

Enable AES-256-GCM in CryptoLib — CM0034 alone blocks Steps 2 and 3. One fix.

PROJECT 1 — RESULTS

LIVE DEMO AVAILABLE

Auto-generated fix. Deployed in under 5 minutes.

AEGIS doesn't produce a findings list. It produces working code.

Before — compromised artifact

```
FAIL — SBOM not verified
FAIL — CryptoLib 1.4.0 vulnerable
FAIL — CVE-2025-29912 present
FAIL — Build provenance missing
FAIL — Wrong signing account

Update accepted — attack succeeds
```

After — AEGIS policy applied

```
PASS — SBOM verified
PASS — CryptoLib 1.4.2 clean
PASS — No known CVEs
PASS — Build provenance confirmed
PASS — HSM signing key verified

Update rejected — attack blocked
```

AEGIS ALSO PRODUCED

4 OPA/Conftest policies
Block supply chain attack at CI/CD gate. Ready to deploy.

7 Sigma detection rules
Vendor-neutral. Splunk, Elastic, Chronicle.

3 Terraform configs
MFA, IAM hardening, encryption. Sprint-ready.

KEY SLIDE

THE FLEET — autonomous multi-agent platform

FLEET GENERAL

The only agent that sees the full picture. Makes live Claude API calls. Reasons over SPARTA v3.2. Assigns tasks. Adapts strategy mid-operation. Never executes attacks directly.

Attack fleet

- **Scout** passive recon
- **Phantom** credentials
- **Breacher** exploitation
- **Wraith** persistence
- **Courier** exfiltration
- **Hammer** active phase

Intel fleet

- **Mapper** architecture
- **Analyst** vulnerabilities
- **Oracle** threat intel

Defense fleet

- **Medic** auto-patch
- **Forge** signing
- **Sentinel** detection
- **Warden** response
- **Judge** compliance

● Built & demonstrated ● Roadmap — completing at Vast

4 agents built. 10 on the roadmap.

The General’s actual decisions. Not scripted.

DECISION 1 — MISSION BRIEF

Crew: NONE

SPARTA: TA0001 Reconnaissance

"Phased approach. Begin recon. Establish full picture before any active steps."

DECISION 2 — AFTER SCOUT REPORT

Crew: MEDIUM

SPARTA: IA-0007 → EX-0009

"Dual-path attack strategy. Primary: IA-0007 ground identity exploiting MFA gaps. Secondary: EX-0009 CryptoLib fallback."

DECISION 3 — AFTER PHANTOM REPORT

Crew: HIGH

SPARTA: IA-0005 → EX-0009

"Signing key obtained. Advance to execution via supply chain. Task Breacher: exploit CVE-2025-29912."

DECISION 4 — AFTER BREACHER REPORT

Crew: CRITICAL

SPARTA: PER-0003 → DE-0003 → IMP-0002

"CRITICAL MILESTONE. Implant confirmed. Task Wraith. Courier on standby. Hammer on standby. Implement strict operational boundaries to prevent crew safety incidents."

NONE → MEDIUM → HIGH → CRITICAL — the General understood the stakes at each phase. Without being told.

What I learned. What I'd do differently.

LESSONS LEARNED

- **Multi-agent coordination is harder than single-agent augmentation.** The General's task assignment logic needed three full rewrites before producing consistent fleet behavior.
- **SPARTA v3.2 coverage is uneven.** OT and CCSDS-specific paths required me to extend the taxonomy with my own mappings.
- **The crew safety flag emerging unprompted raised a harder question.** How do you formally specify safety boundaries for an autonomous agent in a life-critical environment?

WHAT I'D DO DIFFERENTLY

- **Define formal safety constraints before expanding the attack fleet.** Emergent safety reasoning is promising but not sufficient at scale.
- **Build the Intel fleet earlier.** The General's reasoning quality is capped by the richness of Scout's intelligence.
- **Invest in the Judge agent from day one.** Compliance documentation as an automatic byproduct is what makes this viable in a CMMC-regulated environment.

KEY SLIDE

The problem no one has solved yet.

WHY EXISTING APPROACHES FAIL FOR A CREWED SPACE STATION

- Commercial space stations sit at the convergence of **OT (life-safety), IT (corporate/cloud), and AI/ML (autonomy, telemetry)** — no existing red team playbook covers all three simultaneously.
- Signal latency between Haven Station and ground ranges from **250ms to 8 minutes** — a nation-state adversary at machine speed doesn't wait for ground response.
- A crew of 4–6 cannot maintain continuous security monitoring alongside mission operations. **The station needs autonomous detection and response.**
- Standard EDRs reboot to patch. **Haven-1 cannot reboot on demand** — 90 seconds without attitude control means no power, no comms, no life support monitoring.
- Nation-state adversaries are already watching. **Haven-1 launching makes Vast a named target**, not a theoretical one.

MOST IMPORTANT SLIDE

SOVEREIGN

Supreme authority. Self-governance.

A space station that is SOVEREIGN
defends itself autonomously.
Detects threats without ground intervention.
Patches vulnerabilities in orbit.
Cannot be captured, controlled, or coerced
by any adversary.

**Vast's mission is continuous human presence
in space for America and its allies.**

**A SOVEREIGN space station
is one that cannot be taken from them.**

One program. Three tiers. Built incrementally.

TIER 1 · BUILT & RUNNING

AEGIS + THE FLEET

AI-driven SPARTA emulation. Fleet General reasoning over Haven-1. 7-step kill chain. 23 gaps. 47-second runtime. Deployable on day 60.

↓

TIER 2 · POC BUILT · COMPLETING AT VAST

THE FLEET + AIRAG

Full 14-agent autonomous platform. AIRAG integrated into VCAP avionics kernel. Attack and defense run simultaneously. Adapts mid-operation. 2–3 year build.

↓

TIER 3 · THE DESTINATION

SOVEREIGN

The station defends itself. Detects without ground intervention. Patches in orbit without a reboot. Cannot be captured, controlled, or coerced. 1–2 years after The Fleet.

KEY SLIDE

Four capabilities. No ground intervention required.

CIPHER — zero-day discovery

Custom fuzzer targeting CCSDS protocol parsers. AFL++ at 50,000 exec/sec. angr static analysis directs the fuzzer. Z3 theorem prover solves the ROP chain as a constraint problem. Mathematically proves exploitability — not a CVE reference, a working exploit.

THE FLEET + AIRAG — adaptive simulation

14 specialized agents plus AIRAG integrated into the VCAP avionics kernel. Attack and defense run simultaneously. Medic generates patches from live findings. Sentinel calibrates detection rules to observed attack behavior in real time.

PHANTOM REACH — stealth pivot

Full undetected pivot from ground station web portal through high-bandwidth optical link to station life support network. Zero telemetry signature throughout. Standard monitoring sees nothing. AIRAG's physics model sees everything.

GUARDIAN — in-orbit patching

AI-driven RTOS EDR. Detects memory corruption in under 500ms. Synthesizes patch in under 30 seconds. Hot-deploys to running RTOS without a reboot in under 90 seconds total. No crew disruption. No ground intervention. That capability does not exist anywhere else.

SOVEREIGN isn’t built in year one. It’s the destination every decision moves toward.

Days 1–30 **Listen and map**
Understand security posture, asset inventory, CMMC/NIST obligations, engineering relationships. No offensive operations — trust comes before access.

Days 31–60 **First engagement — AEGIS operational**
AEGIS adapted to Vast’s actual AWS environment. First findings delivered. AIRAG architecture scoped against VCAP stack.

Days 61–90 **Program foundation**
Engagement model defined. TTP library v1 drafted. OT/GCS attack surface mapping begun. Red team capability real, documented, repeatable.

Year 1 **Full fleet + AIRAG v1 kernel integration**
Complete attack and defense fleets. AIRAG integrated into VCAP. Physics-as-root-of-trust validation pipeline live.

Year 2–3 **The Fleet + AIRAG at scale**
All 14 agents operational. AIRAG running continuous SPARTA-mapped simulations on-orbit. Binary Specialist and GUARDIAN capabilities proven against Haven-1 digital twin.

Year 4–5 **SOVEREIGN — Haven Station is autonomous**
Full detect-patch-respond cycle. No ground intervention required below crew-safety threshold. Haven Station is SOVEREIGN.

KEY SLIDE

AIRAG

Autonomous Incident Response & AI Guardian — the 5th crew member. Its only job is Haven-1's Digital Immune System.

Vertical integration argument

Because Vast builds its own hardware and software, AIRAG can be baked into the VCAP avionics kernel — not monitoring the network, but monitoring the *voltage of the logic.* If a command to the CMGs doesn't match the physics model of the station, AIRAG kills it in the buffer before hardware moves.

Physics as root of trust

Standard security looks for bad code. AIRAG looks for *bad physics.* By integrating flight dynamics models into the AI, a "software patch" that would cause an unauthorized change in station attitude is recognized as a cyber-kinetic attack. Orbital mechanics is the final firewall.

The 5th crew member

Haven-1 has 4 crew. While they sleep, AIRAG runs continuous SPARTA-mapped simulations using the AEGIS framework to find and patch logic flaws. When the crew wakes up, the attack surface is *smaller* than when they went to bed.

AIRAG is the evolution of AEGIS. It takes the SPARTA reasoning from my laptop and moves it into the station's RTOS. It doesn't just find the gap — it autonomously deploys the OPA gate to close it. In orbit. Without a reboot. Without calling home.

 BRILLIANT IDEA — DEMO THREE-SCREEN DEMO

The Ghost in the Gimbal

A cyber-kinetic attack on Haven-1's Control Moment Gyroscopes — and how AIRAG stops it in 90 seconds.

PHASE 1 — DETECTION VIA THE SOVEREIGN DATA PLANE

Attacker exploits CCSDS parser via compromised research payload

An adversary uses a compromised research payload to exploit a memory corruption vulnerability in the CCSDS parser on the internal flight bus. AIRAG — integrated into the VCAP avionics kernel — detects an unauthorized `write()` attempt to a protected memory region controlling the CMG gimbals. AIRAG queries the SOVEREIGN Fleet General, which uses SPARTA (T1562) to recognize this as the first step of a kinetic exploit chain — not a glitch.

PHASE 2 — AUTONOMOUS TRIAGE (STATION IS ON THE DARK SIDE — NO GROUND LINK)

AIRAG runs a physics simulation using the AEGIS digital twin

AIRAG calculates the requested command would move the station's attitude by 15 degrees in 4 seconds — violating structural hull safety constraints. AIRAG issues an immediate circuit break, virtually air-gapping the compromised research payload from the flight-control bus. Station safe. Attacker blind. Ground hasn't been called yet.

PHASE 3 — GUARDIAN HOT-PATCH (NO REBOOT, NO GROUND INTERVENTION)

Binary Specialist analyzes crash dump — micropatch deployed in 90 seconds

AIRAG's Binary Specialist identifies the exact buffer overflow in the CCSDS parser. Medic generates a micropatch validating input length before it reaches the parser. Forge signs it with station-trusted keys. Patch injected into live RTOS memory. Vulnerability closed. No reboot. No ground packet. Crew never knew.

THE THREE-SCREEN DEMO

Screen 1 — Offense

Binary Specialist output proving the CCSDS buffer overflow exploit works in a controlled lab. Show the working exploit chain.

Screen 2 — AIRAG audit log

```
Anomaly: VCAP Bus 0x04 — unauthorized write()
Physics violation: hull stress > 85%
Active isolation engaged
Patch synthesis: 28 seconds
Patch deployed — no reboot required
```

Screen 3 — AEGIS heatmap

SPARTA heatmap turns from RED (vulnerable) to GREEN (patched) automatically. Gap appeared in real time — closed in real time.

"Haven-1 didn't just survive — it evolved. By the time ground control re-establishes a link, AIRAG has already blocked the attacker, patched the zero-day, and generated a full SPARTA-mapped debrief. We turned a potential mission-loss event into a routine maintenance report."

KEY SLIDE

The numbers make the case.

NASA breach precedent (proven)

ISS near-miss incident costs	$250M+
Mission abort + crew recovery	$180M
Program delay per incident	$400M
Reputational / congressional	$200M+
2020 NASA cyber incidents	~2,000 events
Estimated per major incident	**$1B+**

Vast Haven-1 (estimated)

Crew rescue operation	~$180M
Vehicle loss	~$400M
Liability + legal	~$350M
Reputational damage	~$200M
Program valuation impact	~$500M+
Total estimated exposure	**~$2.1B**

AEGIS + AIRAG + SOVEREIGN — year one

Development	$180K
Claude API usage	$8K
Infrastructure	$24K
Engineer FTE (this role)	$280K
Maintenance	$40K
Total year one	**$532K**

~3,700x
ROI on first incident prevented

8 min
Ground signal delay to Haven-1

90 sec
AIRAG autonomous response time

NASA cost figures from publicly available NASA OIG and GAO reports. Vast figures are engineering estimates.

Haven-1 carries **four crew.**

Haven-2 carries **eight.**

The Artificial Gravity Station carries **forty permanently.**

The cost of getting this wrong

scales with every person we put in space.

The cost of **SOVEREIGN** does not.

AEGIS is day one. AIRAG is the mission.
The work is already started. Let's finish it.

MY QUESTIONS FOR YOU

- "What does the security team's relationship with engineering look like today — are you pulled in early on design decisions, or mostly reactive after the fact?"
- "What's the biggest security concern as Haven-1 goes live — the corporate environment, the mission systems, or the boundary between them?"
- "When visiting vehicles dock with Haven-1, what does the current security model look like for that inter-vehicle network boundary? Is there a formal specification for what the station is and is not permitted to trust from a docked vehicle — their firmware, their data buses, their physical access to internal systems?"

BACKUP SLIDES AVAILABLE ON REQUEST

- AI Kill-Chain Emulator for Cloud & Container Security — GitHub portfolio project, goals / contributions / results / lessons
- AIRAG technical deep-dive — full Ghost in the Gimbal architecture, RTOS integration model, formal safety constraint specification